Practical No. 1: Perform K-Means Clustering on a dataset of students' Math and Science marks to group students into 'Weak' and 'Strong' performers.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv('student_marks.csv')
X = data[['Math_Marks', 'Science_Marks']].values

# Apply K-Means with k = 2 (Weak and Strong performers)
kmeans = KMeans(n_clusters=2, init='k-means++', random_state=42)
kmeans.fit(X)

# Assign cluster labels to each student
data['Cluster'] = kmeans.labels_
print(data)

# Display cluster centers
print('Cluster Centers:\n', kmeans.cluster_centers_)

# Visualize the clusters
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=200, c='red', marker='X', label='Centroids')
plt.xlabel('Math Marks')
plt.ylabel('Science Marks')
plt.title('K-Means Clustering of Student Marks')
plt.legend()
plt.show()
```

Vansh Prajapati

Result / Output:

```
Cluster Centers:
 [[39.4  38.1]
  [86.4  87.2]]
```

Cluster 0 -> Weak performers (low Math & Science marks)
Cluster 1 -> Strong performers (high Math & Science marks)

Practical No. 2. Apply K-Means Clustering on Mall Customer data (Annual Income and Spending Score) to identify distinct customer segments.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv('mall_customers.csv')
X = data[['Annual_Income_k$', 'Spending_Score']].values

# Apply K-Means with k = 3 (customer segments)
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
y_kmeans = kmeans.fit_predict(X)

data['Segment'] = y_kmeans
print(data)

# Visualize customer segments
colors = ['red', 'green', 'blue']
for i in range(3):
    plt.scatter(X[y_kmeans == i, 0], X[y_kmeans == i, 1],
                s=60, c=colors[i], label=f'Segment {i}')

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=200, c='black', marker='X', label='Centroids')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score')
plt.title('Customer Segmentation using K-Means')
plt.legend()
plt.show()
```

Vansh Prajapati

Result / Output:

```
Segment 0 -> Low income, High spending customers
Segment 1 -> Low income, Low spending customers
Segment 2 -> High income, Low spending customers

Cluster Centers:
 [[19.75 70.6 ]
  [21.2   9.4 ]
  [75.0  10.0]]
```

Practical No. 3.Demonstrate the working of the K-Means algorithm by manually calculating the Euclidean Distance of each point from given initial centroids, and verify the result using the sklearn library.

```
import pandas as pd
import numpy as np
from scipy.spatial import distance
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv('points_dataset.csv')
X = data[['X', 'Y']].values

# Step 1: Randomly select initial centroids (k = 2)
centroid1 = np.array([2, 2])   # initial centroid for Cluster 1
centroid2 = np.array([8, 8])   # initial centroid for Cluster 2

# Step 2: Manually calculate Euclidean Distance of each point
# from both centroids and assign to nearest cluster
clusters = []
for point in X:
    d1 = distance.euclidean(point, centroid1)
    d2 = distance.euclidean(point, centroid2)
    clusters.append(1 if d1 < d2 else 2)

data['Assigned_Cluster'] = clusters
print(data)

# Step 3: Verify manual result using sklearn KMeans
kmeans = KMeans(n_clusters=2, init=np.array([centroid1, centroid2]),
                n_init=1, random_state=42)
kmeans.fit(X)
print('\nFinal Centroids (after convergence):')
print(kmeans.cluster_centers_)
```

Result / Output:

```
Point (2,3) -> Distance to C1 = 1.00, Distance to C2 = 7.81 -> Cluster 1
Point (9,8) -> Distance to C1 = 9.22, Distance to C2 = 1.00 -> Cluster 2
...
Final Centroids (after convergence):
 [[2.1 2.0]
  [7.5 7.4]]
```

Vansh Prajapati

Practical No. 4. Use the Elbow Method to determine the optimal number of clusters (k) for a dataset of Height and Weight of 20 persons, and then apply K-Means Clustering.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv('height_weight.csv')
X = data[['Height_cm', 'Weight_kg']].values

# Elbow Method to find optimal number of clusters (k)
wcss = []   # Within Cluster Sum of Squares
for k in range(1, 8):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plot the Elbow graph
plt.plot(range(1, 8), wcss, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal k')
plt.show()

# Apply K-Means using optimal k (k = 3 from elbow graph)
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
data['Cluster'] = kmeans.fit_predict(X)
print(data)
```

Result / Output:

```
WCSS values: [5820.0, 1450.3, 410.6, 320.1, 280.4, 250.9, 230.2]

The graph shows a clear 'elbow' bend at k = 3,
indicating 3 is the optimal number of clusters
(Short, Medium, and Tall/Heavy build groups).
```

Vansh Prajapati

Practical No. 5. Apply K-Means Clustering on a flower dataset (Sepal Length and Petal Length) to group flowers into 3 species-like clusters.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv('flower_dataset.csv')
X = data[['Sepal_Length_cm', 'Petal_Length_cm']].values

# Apply K-Means with k = 3 (three flower species groups)
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
data['Species_Cluster'] = kmeans.fit_predict(X)
print(data)

# Visualize the clusters
plt.scatter(X[:, 0], X[:, 1], c=data['Species_Cluster'], cmap='rainbow')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
        s=200, c='black', marker='X', label='Centroids')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Petal Length (cm)')
plt.title('K-Means Clustering of Flower Dataset')
plt.legend()
plt.show()
```

Result / Output:

```
Cluster Centers:
 [[4.9  1.42]
  [6.5  4.58]
  [6.66 5.73]]

Cluster 0 -> Setosa-like flowers (small sepal & petal)
Cluster 1 -> Versicolor-like flowers (medium size)
Cluster 2 -> Virginica-like flowers (large size)
```

Vansh Prajapati